

Variational Solver Flow Matching for Forward Dynamics Generation

Abstract

Forward dynamics prediction is a central problem in machine learning, robotics, and world modeling. Given past observations and actions, the goal is to predict the next state or future trajectory. Existing approaches often learn either deterministic transition functions or generic generative models in latent space. While expressive, such models typically do not enforce that predicted transitions satisfy a principled dynamical law. In this proposal, we develop a new model family called *Variational Solver Flow Matching* (VSFM), which recasts next-state prediction as the numerical solution of a learned discrete Euler–Lagrange equation and uses flow matching to amortize that solve. Concretely, we learn a continuous latent state $q_t = E_\phi(o_t)$, a scalar-valued discrete Lagrangian $L_{d,\theta}(q_t, q_{t+1}, a_t, c_t)$, and a solver-time vector field that transports a noisy candidate next state toward the solution of the induced discrete variational equation. The resulting model is not merely physics-inspired: it is explicitly grounded in the relationship between the classical action principle, the discrete Euler–Lagrange equation, and numerical root-finding. This gives a mathematically consistent bridge between Lagrangian mechanics and flow matching for forward dynamics generation.

1 Introduction

Forward dynamics prediction aims to model

$$p(x_{t+1} \mid x_t, a_t) \quad \text{or} \quad p(x_{t+1:t+H} \mid x_{0:t}, a_{t:t+H-1}),$$

where x_t denotes the system state and a_t denotes the control input. In robotics, these dynamics govern future physical evolution. In video world models, they govern the evolution of latent scene states. In either case, accurate next-state prediction is fundamental for control, planning, and simulation.

Most current approaches fall into one of two categories. The first learns a deterministic transition function of the form

$$x_{t+1} = f_\theta(x_t, a_t).$$

The second learns a conditional generative model in latent space, such as a diffusion or flow-based model, to represent multimodal future evolution. These methods are often powerful in practice, but they share a major limitation: they usually treat dynamics as an arbitrary conditional mapping rather than as the outcome of an underlying variational law.

Lagrangian mechanics provides a principled view of dynamics. In the classical formulation, system trajectories are stationary points of an action functional. This yields the Euler–Lagrange equation, which constrains the lawful evolution of the state. However, classical mechanics is usually formulated in continuous time, while learning problems are posed with discrete observations. If one starts with the continuous Euler–Lagrange equation and discretizes afterward, prediction error becomes entangled with numerical integration error. For next-state prediction, the proper object is not continuous-time simulation first and learning second, but rather a *discrete-time variational law* that directly constrains adjacent states.

This proposal develops a new model family called *Variational Solver Flow Matching* (VSFM). The key idea is to interpret next-state prediction as the numerical solution of a *learned discrete Euler–Lagrange equation*, and then use *flow matching as an amortized solver* for that nonlinear variational equation. This gives the following conceptual chain:

continuous action principle $\longrightarrow$ discrete Euler–Lagrange equation $\longrightarrow$ nonlinear next-state solve $\longrightarrow$ flow-mat

The resulting model is a mathematically coherent synthesis of variational mechanics, numerical solving, and generative transport.

2 Problem Formulation

Let o_t denote the observation at time t , such as an image, point cloud, proprioceptive state, or multimodal observation, and let a_t denote the action. We define a learned latent state

$$q_t = E_\phi(o_t), \quad q_t \in \mathbb{R}^d.$$

The goal is to predict the next latent state q_{t+1} conditioned on recent latent history and actions:

$$q_{t+1} \sim p_\Theta(q_{t+1} \mid q_{t-1}, q_t, a_{t-1}, a_t, c_{t-1}, c_t),$$

where c_t denotes optional context such as task embeddings, object identity, scene embeddings, or language conditioning.

If desired, the predicted latent state can be decoded back into observation space:

$$\hat{o}_{t+1} = D_\omega(q_{t+1}).$$

We focus first on one-step prediction, since it yields the cleanest variational formulation. Multi-step rollout can then be obtained recursively.

3 Relationship to the Lagrangian Equation

3.1 Continuous-Time Lagrangian Mechanics

In classical mechanics, a system trajectory $q(t)$ is governed by a Lagrangian

$$L(q, \dot{q}, t) = T(q, \dot{q}, t) - V(q, t),$$

where T is kinetic energy and V is potential energy. The trajectory is a stationary point of the action

$$\mathcal{S}[q] = \int_{t_0}^{t_1} L(q(t), \dot{q}(t), t) dt.$$

Taking the first variation of $\mathcal{S}$ yields the Euler–Lagrange equation

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}} - \frac{\partial L}{\partial q} = 0.$$

This equation is the continuous-time law of motion. However, in learning problems we do not observe a continuous path; we observe samples (q_t) at discrete times. Moreover, the task is not to simulate an already known ODE, but to *predict the next sampled state*. Therefore, the proper variational object is the discrete-time analogue of the Lagrangian formulation.

3.2 Discrete Variational Mechanics

Instead of a continuous-time Lagrangian density, define a *discrete Lagrangian*

$$L_d(q_t, q_{t+1}, a_t, c_t),$$

which approximates the action accumulated over a single time step. Intuitively, while the continuous Lagrangian assigns cost density to an infinitesimal motion, the discrete Lagrangian assigns action directly to a pair of adjacent states.

The discrete action over a trajectory $q_0, \dots, q_T$ is

$$\mathcal{S}_d(q_{0:T}) = \sum_{t=0}^{T-1} L_d(q_t, q_{t+1}, a_t, c_t).$$

A discrete trajectory is a stationary point of $\mathcal{S}_d$ if and only if it satisfies the discrete Euler–Lagrange (DEL) equation

$$D_2 L_d(q_{t-1}, q_t, a_{t-1}, c_{t-1}) + D_1 L_d(q_t, q_{t+1}, a_t, c_t) = 0,$$

where D_1 and D_2 denote derivatives with respect to the first and second state arguments, respectively.

This is the discrete-time counterpart of the continuous Euler–Lagrange equation. Therefore, the relationship between the proposed model and Lagrangian mechanics is exact:

- the continuous action principle yields the Euler–Lagrange equation,
- the discrete action principle yields the discrete Euler–Lagrange equation,
- next-state prediction in sampled data can be written as solving the DEL equation for the unknown q_{t+1} .

This last point is critical. Given $(q_{t-1}, q_t, a_{t-1}, a_t)$, the next state is not arbitrary: it must satisfy

$$D_2 L_d(q_{t-1}, q_t, a_{t-1}, c_{t-1}) + D_1 L_d(q_t, q_{t+1}, a_t, c_t) = 0.$$

Hence, next-state prediction is fundamentally a *nonlinear variational solve*.

4 Learned Discrete Lagrangian

We parameterize the discrete Lagrangian using a neural network:

$$L_{d,\theta}(q_t, q_{t+1}, a_t, c_t) \in \mathbb{R}.$$

This model is intentionally simple: it is a scalar-valued network that directly learns the discrete action between adjacent latent states. We do not explicitly separate kinetic and potential energy. Instead, the network learns the variational structure in latent space.

For a candidate next state y , define the discrete Euler–Lagrange residual

$$R_\theta(y; \xi_t) = D_2 L_{d,\theta}(q_{t-1}, q_t, a_{t-1}, c_{t-1}) + D_1 L_{d,\theta}(q_t, y, a_t, c_t),$$

where

$$\xi_t = (q_{t-1}, q_t, a_{t-1}, a_t, c_{t-1}, c_t).$$

The correct next state should satisfy

$$R_\theta(q_{t+1}; \xi_t) \approx 0.$$

Thus, next-state prediction becomes

$$\text{find } y \text{ such that } R_\theta(y; \xi_t) = 0.$$

This is the exact point where the learned discrete Lagrangian and the prediction problem become identical: the model does not merely regularize predictions; it defines the nonlinear equation those predictions should satisfy.

5 From Numerical Solution to Flow Matching

5.1 Variational Solver Energy

To solve the residual equation $R_\theta(y; \xi_t) = 0$, define the energy

$$\Phi_\theta(y; \xi_t) = \frac{1}{2} \|R_\theta(y; \xi_t)\|^2.$$

The true next state is ideally a zero-residual minimizer:

$$\Phi_\theta(q_{t+1}; \xi_t) = 0.$$

A standard numerical method would update y by descending Φ_θ :

$$y \leftarrow y - \eta \nabla_y \Phi_\theta(y; \xi_t).$$

Since

$$\nabla_y \Phi_\theta(y; \xi_t) = J_R(y; \xi_t)^\top R_\theta(y; \xi_t),$$

where J_R is the Jacobian of R_θ with respect to y , this provides a principled solver direction.

5.2 Continuous Solver-Time ODE

Rather than applying a fixed finite number of hand-designed optimization steps, we define a continuous solver-time dynamics

$$\frac{dy_s}{ds} = -P_\psi(y_s, s, \xi_t) \nabla_y \Phi_\theta(y_s; \xi_t), \quad s \in [0, 1].$$

Here:

- s is *solver time*, not physical time,
- $P_\psi(y_s, s, \xi_t) \succ 0$ is a learned preconditioner or metric,
- the flow starts from a noisy initial guess and refines it into a valid next state.

This is already a meaningful solver. The next step is to train this solver-time dynamics using flow matching.

6 Variational Solver Flow Matching

6.1 Conditional Path

Let

$$z := q_{t+1}$$

be the true next latent state, and sample Gaussian noise

$$\epsilon \sim \mathcal{N}(0, I).$$

Define the interpolation path

$$y_s = (1 - s)\epsilon + sz, \quad s \in [0, 1].$$

Its conditional target field is

$$u^*(y_s; z, \epsilon) = z - \epsilon.$$

This is the standard conditional flow-matching target for a linear path from Gaussian noise to a data point.

6.2 Solver-Structured Flow Field

A generic flow-matching model would predict an unconstrained vector field $u_\psi(y_s, s, \xi_t)$. In contrast, we define the field using the numerical solution structure of the discrete Lagrangian equation:

$$u_{\theta, \psi}(y, s, \xi_t) = -\alpha(s) P_\psi(y, s, \xi_t) \nabla_y \Phi_\theta(y; \xi_t) + \beta(s) w_\psi(y, s, \xi_t).$$

Here:

- the first term is the *variational solver term*,
- the second term is an optional *residual correction field*,
- $\alpha(s)$ and $\beta(s)$ are schedules controlling the contributions of the two terms.

The first term carries the core structure: it is induced directly by the discrete Euler–Lagrange residual energy. The second term captures model mismatch, multimodality, partial observability, or latent compression effects.

In the minimal formulation, we set $\beta(s) = 0$, yielding

$$u_\theta(y, s, \xi_t) = -\alpha(s) \nabla_y \Phi_\theta(y; \xi_t).$$

This is the cleanest and most structured version of the model.

7 Training Objective

The model is trained with three coupled losses.

7.1 Flow-Matching Solver Loss

We match the predicted solver field to the conditional flow-matching target:

$$\mathcal{L}_{\text{VSFM}} = \mathbb{E}_{s, \epsilon} \left[\|u_{\theta, \psi}(y_s, s, \xi_t) - (z - \epsilon)\|^2 \right].$$

This trains the solver-time ODE to transport a noisy candidate toward the correct next latent state.

7.2 Discrete Variational Consistency Loss

We also require the true next state to satisfy the learned discrete Euler–Lagrange law:

$$\mathcal{L}_{\text{DEL}} = \|R_\theta(z; \xi_t)\|^2.$$

This ensures that the learned discrete Lagrangian is meaningful and that its induced equation is compatible with the observed data.

7.3 Representation or Reconstruction Loss

If the latent must decode back into observation space, we add

$$\hat{o}_{t+1} = D_\omega(z),$$

with reconstruction loss

$$\mathcal{L}_{\text{rec}} = \|D_\omega(z) - o_{t+1}\|^2.$$

Alternative latent consistency losses may also be used.

7.4 Total Objective

The full training objective is

$$\mathcal{L} = \lambda_{\text{VSFM}} \mathcal{L}_{\text{VSFM}} + \lambda_{\text{DEL}} \mathcal{L}_{\text{DEL}} + \lambda_{\text{rec}} \mathcal{L}_{\text{rec}}.$$

This objective has a direct interpretation:

- $\mathcal{L}_{\text{DEL}}$ learns the discrete dynamics law,
- $\mathcal{L}_{\text{VSFM}}$ learns an amortized solver for that law,
- $\mathcal{L}_{\text{rec}}$ stabilizes the latent representation and supports decoding.

8 Inference

At inference time, given $(o_{t-1}, o_t, a_{t-1}, a_t)$, we compute

$$q_{t-1} = E_\phi(o_{t-1}), \quad q_t = E_\phi(o_t).$$

We initialize a noisy candidate next state

$$y_0 \sim \mathcal{N}(0, I),$$

and solve the solver-time ODE

$$\frac{dy_s}{ds} = u_{\theta, \psi}(y_s, s, \xi_t), \quad s : 0 \rightarrow 1.$$

The final state is the prediction

$$\hat{q}_{t+1} = y_1.$$

If needed, decode

$$\hat{o}_{t+1} = D_\omega(\hat{q}_{t+1}).$$

Thus, inference is literally a learned numerical solve in solver time.

9 Interpretation as a Learned Variational Integrator

A classical variational integrator:

1. defines a discrete Lagrangian,
2. solves the discrete Euler–Lagrange equation for the next state.

The proposed model follows the same pattern, except that:

- the discrete Lagrangian is learned in latent space,
- the DEL equation is solved using a learned flow-matching solver instead of a hand-designed Newton or fixed-point method.

Therefore, the precise mathematical relationship between the model and the Lagrangian equation is:

continuous EL $\longrightarrow$ discrete EL $\longrightarrow$ root-finding problem $\longrightarrow$ solver-time flow matching.

This chain is the core theoretical contribution of the proposal.

10 Multi-Step Extension

The one-step solver can be unrolled autoregressively:

$$\hat{q}_{t+1} = \text{Solve}(\hat{q}_{t-1}, \hat{q}_t, a_{t-1}, a_t),$$

$$\hat{q}_{t+2} = \text{Solve}(\hat{q}_t, \hat{q}_{t+1}, a_t, a_{t+1}),$$

and so on.

One may also define a full-horizon discrete action

$$\mathcal{S}_d(q_{t:t+H}) = \sum_{k=t}^{t+H-1} L_{d,\theta}(q_k, q_{k+1}, a_k, c_k),$$

and construct a trajectory-space solver flow for joint prediction of the entire future rollout. However, the one-step formulation is the correct starting point because it preserves the clearest relationship between the next-state prediction problem and the discrete Euler–Lagrange equation.

11 Expected Advantages

The proposed model offers several advantages.

First, it provides a strong inductive bias: predicted next states are constrained by a learned variational law rather than by an unconstrained conditional generator.

Second, it matches the discrete nature of the observed data. The model is discrete in physical time and therefore does not mix learning error with an external integration scheme.

Third, it assigns a precise scientific role to flow matching. Flow matching is not used merely as a generic generator, but as an *amortized nonlinear solver*.

Fourth, it encourages the latent representation to become dynamics-aware, since the encoder must produce a latent state that supports a discrete variational update law.

12 Risks and Limitations

The main challenge is optimization cost. Since R_θ depends on derivatives of the learned discrete Lagrangian, and $\nabla_y \Phi_\theta$ introduces additional differentiation, training may require higher-order gradients.

A second challenge is residual dominance. If the residual field w_ψ is too expressive, it may absorb most of the predictive burden and weaken the influence of the variational term. Therefore, the minimal model without w_ψ should be tested first.

A third challenge is representation collapse. If the latent state does not preserve enough dynamical information, the learned discrete Lagrangian may capture spurious patterns. Reconstruction losses or multi-step consistency losses can help mitigate this issue.

13 Minimal Implementation Plan

The cleanest initial implementation uses:

$$\begin{aligned} q_t &= E_\phi(o_t), \\ L_{d,\theta}(q_t, q_{t+1}, a_t, c_t) &\in \mathbb{R}, \\ R_\theta(y; \xi_t) &= D_2 L_{d,\theta}(q_{t-1}, q_t, a_{t-1}, c_{t-1}) + D_1 L_{d,\theta}(q_t, y, a_t, c_t), \\ \Phi_\theta(y; \xi_t) &= \frac{1}{2} \|R_\theta(y; \xi_t)\|^2, \\ u_\theta(y, s, \xi_t) &= -\alpha(s) \nabla_y \Phi_\theta(y; \xi_t). \end{aligned}$$

The corresponding loss is

$$\mathcal{L} = \lambda_{\text{VSFM}} \mathbb{E} [\|u_\theta(y_s, s, \xi_t) - (z - \epsilon)\|^2] + \lambda_{\text{DEL}} \|R_\theta(z; \xi_t)\|^2 + \lambda_{\text{rec}} \mathcal{L}_{\text{rec}}.$$

This version is already structured, novel, and implementable.

14 Conclusion

This proposal introduces *Variational Solver Flow Matching*, a forward dynamics model grounded in the discrete Lagrangian equation. The key idea is:

Predicting the next state is solving a learned discrete Euler–Lagrange equation, and flow matching is used to an

The relationship to Lagrangian mechanics is exact. The continuous action principle yields the Euler–Lagrange equation; the discrete action principle yields the discrete Euler–Lagrange equation; the latter defines a nonlinear root-finding problem for the next state; and the proposed flow-matching model learns solver-time dynamics that transport a noisy candidate toward that root.

This gives a mathematically consistent fusion of variational mechanics, numerical solving, and flow matching, and opens a new direction for physically structured forward dynamics generation in latent space.